

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 5

Comprehensive Study of CNN Training, Regularization, Optimization, Hyperparameter Tuning, Transfer Learning and Cross-Validation

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to systematically study the effect of weight initialization, regularization, optimization algorithms, CNN hyperparameters, transfer learning, fine-tuning and cross-validation on image classification performance.

The experiment uses a single CNN architecture, MobileNetV2, and the Oxford-IIIT Pet dataset. Students will analyze the effect of individual design choices using training/validation curves and finally select a suitable configuration using 5-fold cross-validation.

2. Learning Outcomes

After completing this experiment, students will be able to:

- explain different weight initialization strategies;
- identify and analyze overfitting using training and validation curves;
- compare SGD, Momentum, RMSProp and Adam;
- explain CNN hyperparameters such as kernel size, stride, padding and pooling;
- understand Batch Normalization and Dropout;
- perform transfer learning and fine-tuning using MobileNetV2;
- tune important training hyperparameters;
- apply 5-fold cross-validation for model selection; and
- justify the final model using accuracy, variability and computational cost.

3. Source Code

https://github.com/SamrithaShree/Deep_Learning/tree/main/Lab-5

4. Dataset and Experimental Setup

Dataset: Oxford-IIIT Pet Dataset

The Oxford-IIIT Pet Dataset contains images of cats and dogs belonging to 37 breeds. The images are RGB and have different spatial dimensions. For this experiment, all images are resized to

$$224 \times 224 \times 3.$$

The dataset is suitable for demonstrating transfer learning using ImageNet-pretrained CNNs.

Experimental considerations:

- Use RGB images resized to 224×224 .
- Normalize the input images according to the pretrained model requirements.
- Maintain separate training, validation and test data.
- The test set must remain untouched during hyperparameter selection.
- Use MobileNetV2 because it is computationally lighter than many large CNN architectures and is more suitable for CPU-based laboratory work.

Data split actually used: the provided **train** split (3680 images) was divided into 2944 training and 736 validation images (80/20), and the provided **test** split (3669 images) was retained as the independent test set, untouched until Section 12.



Figure 1: Sample images from the Oxford-IIIT Pet Dataset (37 breeds).

Inference: The samples show that the task is fine-grained – several breeds differ only in coat texture or colour – and that images vary widely in pose, scale, background and lighting. This is why training choices such as initialization, regularization and optimizer actually matter on this dataset.

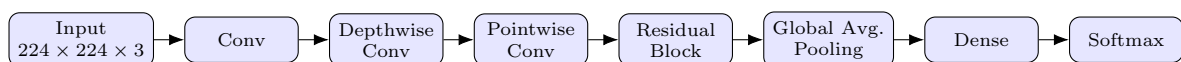
5. MobileNetV2 Architecture

MobileNetV2 is a lightweight CNN architecture designed to reduce computational complexity while maintaining good image representation.

Its important components include:

- depthwise convolution;
- pointwise 1×1 convolution;
- inverted residual blocks;
- linear bottlenecks;
- batch normalization; and
- ReLU6 activation.

A simplified representation is:



Note: The above diagram is a simplified conceptual representation rather than the complete MobileNetV2 architecture.

6. Weight Initialization

Weight initialization determines the initial values of trainable weights before training begins. A suitable initialization can improve convergence and reduce problems such as vanishing or exploding activations.

Students should investigate:

- Zero initialization
- Random initialization
- Xavier/Glorot initialization
- He initialization

Expected Analysis

Students should compare the initialization strategies using:

Plot 1: Training Loss vs. Epoch

- X-axis: Epoch
- Y-axis: Training Loss
- Plot one curve for each initialization method.

Plot 2: Validation Accuracy vs. Epoch

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Plot one curve for each initialization method.

Students should identify which initialization gives faster and more stable convergence.

Results (MobileNetV2 from scratch, Adam, lr = 0.001, 5 epochs)

Initialization	Final Training Loss	Final Val. Accuracy	Time
Zero	3.61	1.49%	2.5 min
Random (unscaled Normal, std = 1.0)	5.37	3.12%	3.6 min
Xavier/Glorot	2.94	2.72%	2.2 min
He	2.78	2.72%	2.0 min

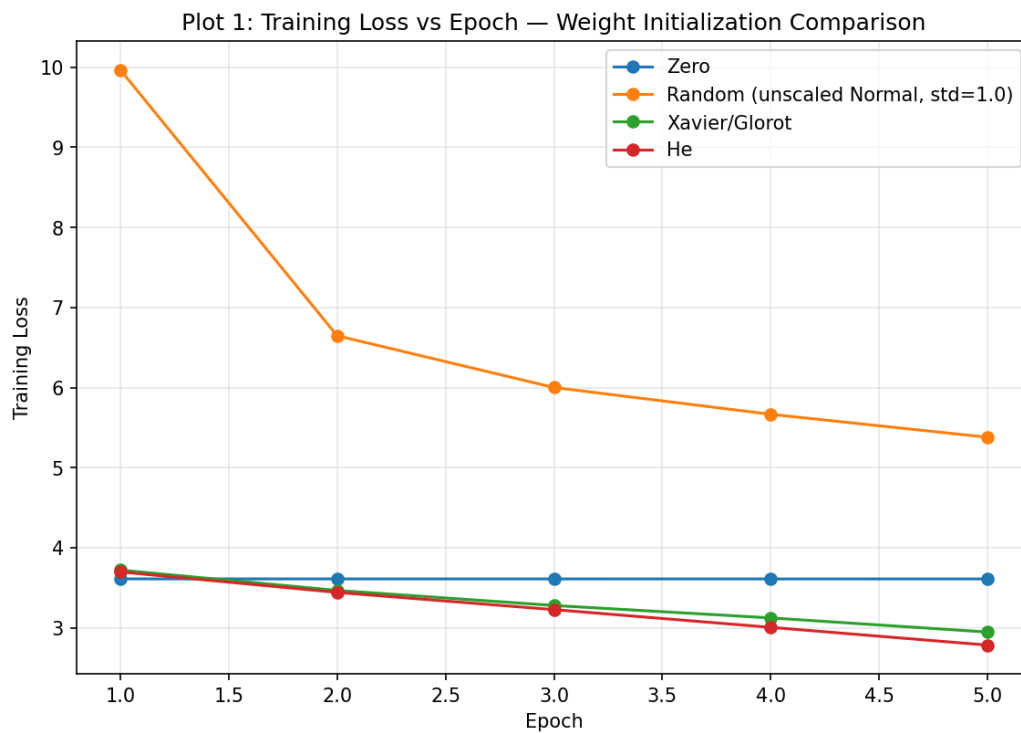


Figure 2: Plot 1: Training Loss vs. Epoch – weight initialization comparison.

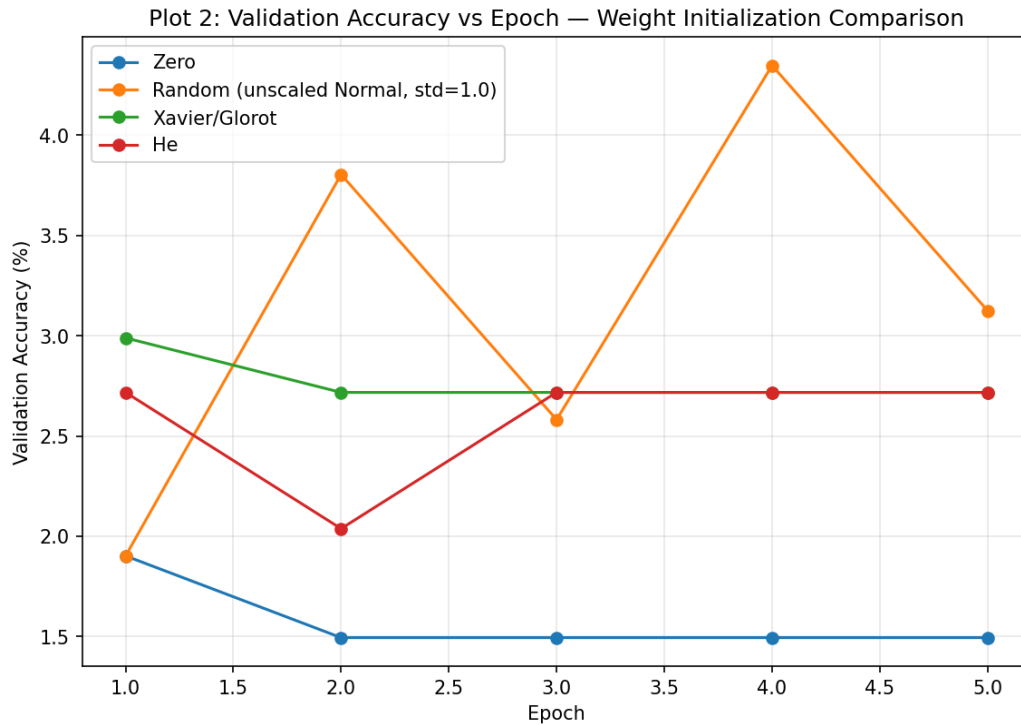


Figure 3: Plot 2: Validation Accuracy vs. Epoch – weight initialization comparison.

Inference (Plots 1 & 2): The plots show how training loss and validation accuracy evolve for four initialization schemes. Zero initialization stays frozen at a loss of $\approx 3.61 = \ln(37)$ with accuracy stuck at chance (1.49%), because identical weights give every neuron identical gradients and symmetry is never broken. Unscaled Random initialization starts at a very high loss (≈ 9.95) and decreases only slowly and noisily, since $\text{std} = 1.0$ makes activations explode through MobileNetV2's depth. Xavier and He both fall smoothly, with He reaching the lowest final loss (2.78) in the least time, as expected for a ReLU6-based network.

7. Regularization and Overfitting

Overfitting occurs when a model performs very well on the training data but generalizes poorly to unseen data.

Students should compare:

- No regularization
- L2 regularization
- Dropout
- Batch Normalization

Expected Plots

Plot 3: Training and Validation Accuracy vs. Epoch

- X-axis: Epoch
- Y-axis: Accuracy (%)
- Plot separate curves for training and validation accuracy.

Students should identify the generalization gap.

Plot 4: Training and Validation Loss vs. Epoch

- X-axis: Epoch
- Y-axis: Loss
- Plot training and validation loss.

Students should identify whether validation loss begins to increase while training loss continues to decrease.

Results (He-initialized MobileNetV2 from scratch, Adam, 6 epochs)

Regularization	Final Train Acc.	Final Val. Acc.	Generalization Gap
No Regularization	14.10%	2.72%	11.38 pp
L2 Regularization	11.89%	2.85%	9.04 pp
Dropout (0.5)	3.91%	2.04%	1.87 pp
Batch Normalization (in head)	19.97%	2.45%	17.53 pp



Figure 4: Plot 3: Training vs. Validation Accuracy – no regularization (baseline).

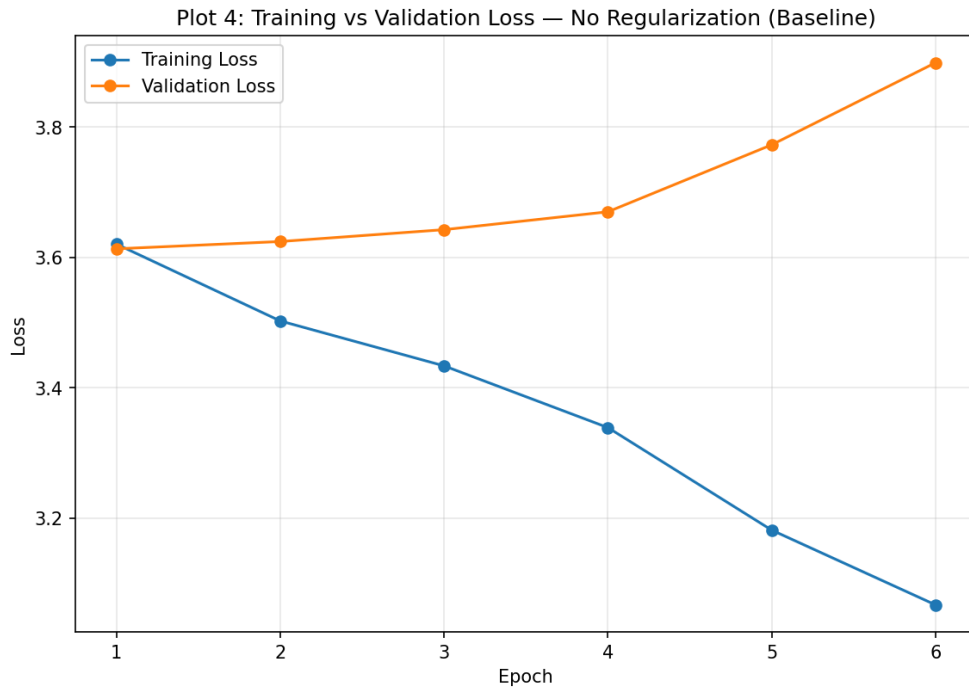


Figure 5: Plot 4: Training vs. Validation Loss – no regularization (baseline).

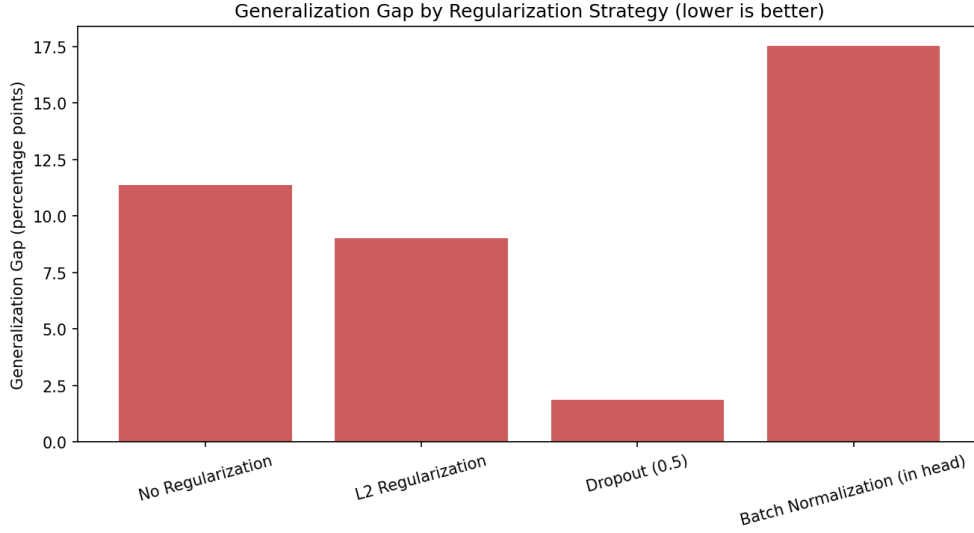


Figure 6: Generalization gap by regularization strategy (lower is better).

Inference (Plots 3 & 4): Plot 3 shows training accuracy rising steadily from 4.3% to 14.1% while validation accuracy stays flat near 2.7%, opening a generalization gap of 11.38 percentage points. Plot 4 shows the matching loss signature: training loss falls from 3.62 to 3.06 while validation loss *rises* from 3.61 to 3.90. This is textbook overfitting – the network is memorising training examples rather than learning transferable features, which is unsurprising for a 2.3M-parameter network trained from scratch on only 2944 images. The summary bar chart confirms Dropout (0.5) produces the smallest gap (1.87 pp), while adding Batch Normalization to the head actually increases the gap (17.53 pp) because it accelerates fitting the training set.

8. Batch Normalization

Batch Normalization normalizes activations within a mini-batch during training.

For a mini-batch

$$x_1, x_2, \dots, x_m,$$

the batch mean is

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i$$

and the batch variance is

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2.$$

The normalized activation is

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}.$$

The final output is

$$y_i = \gamma \hat{x}_i + \beta,$$

where γ and β are learnable parameters.

Numerical Example

Consider

$$x = [2, 4, 6, 8].$$

The mean is

$$\mu_B = \frac{2 + 4 + 6 + 8}{4} = 5.$$

The variance is

$$\sigma_B^2 = \frac{(2-5)^2 + (4-5)^2 + (6-5)^2 + (8-5)^2}{4} = 5.$$

Therefore,

$$\sqrt{5} \approx 2.236.$$

Ignoring the very small ϵ ,

$$\hat{x}_1 = \frac{2-5}{2.236} \approx -1.342,$$

$$\hat{x}_2 = \frac{4-5}{2.236} \approx -0.447,$$

$$\hat{x}_3 = \frac{6-5}{2.236} \approx 0.447,$$

$$\hat{x}_4 = \frac{8-5}{2.236} \approx 1.342.$$

Hence,

$$\hat{x} \approx [-1.342, -0.447, 0.447, 1.342]$$

If $\gamma = 1$ and $\beta = 0$, these are the final outputs.

Inference: Batch Normalization produces approximately zero-centred and unit-scaled activations for this mini-batch. The learnable γ and β allow the network to learn an appropriate scale and shift.

Verification in code: the Keras `BatchNormalization` layer returned $[-1.3416, -0.4472, 0.4472, 1.3416]$ for the same input with $\gamma = 1, \beta = 0$, matching the hand calculation exactly.

Plot 5: With vs. Without Batch Normalization

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Compare two curves: With BN and Without BN.

Configuration	Best Val. Accuracy	Final Val. Accuracy
Without BN	3.53%	2.58%
With BN	3.40%	2.17%

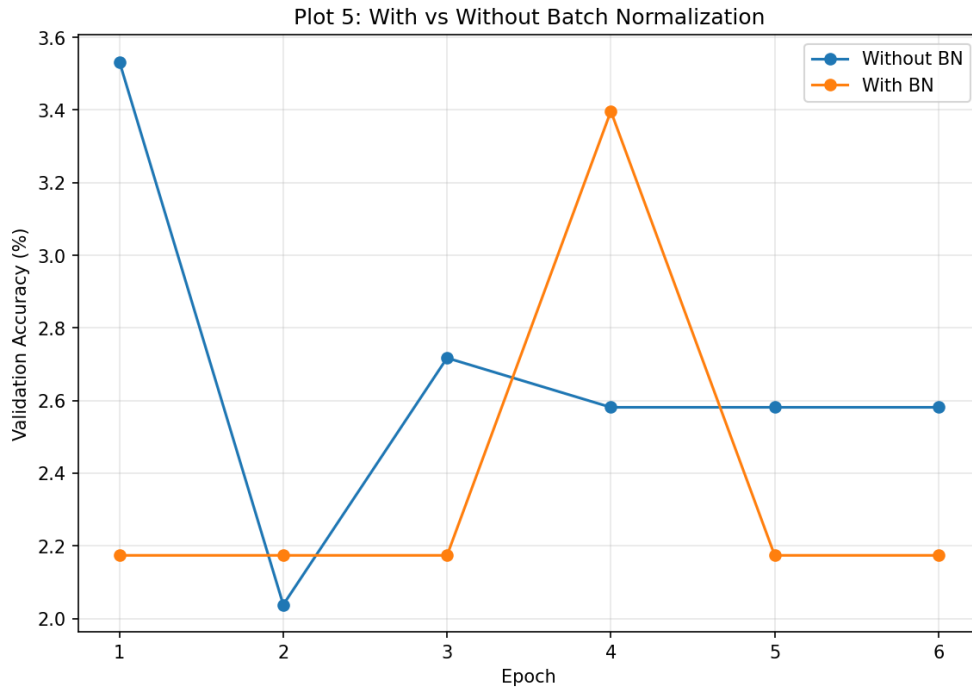


Figure 7: Plot 5: With vs. Without Batch Normalization – validation accuracy.

Inference (Plot 5): The plot compares validation accuracy over six epochs with and without a BN layer in the classifier head. Both curves stay close to chance level ($\approx 2.7\%$) and neither shows a clear advantage, because the dominant bottleneck here is the randomly initialized convolutional base – a single BN layer in the head cannot compensate for a feature extractor that has not yet learned useful filters. The BN curve is flatter across most epochs, consistent with BN’s stabilising role, but the benefit only becomes visible when the base itself is trainable or pretrained (Section 10).

9. Optimization Algorithms

Students should compare the following optimization methods:

- SGD
- Momentum
- RMSProp
- Adam

The objective is to study convergence speed, stability and final validation performance.

Plot 6: Training Loss vs. Epoch for Different Optimizers

- X-axis: Epoch
- Y-axis: Training Loss
- One curve for each optimizer.

Plot 7: Validation Accuracy vs. Epoch for Different Optimizers

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- One curve for each optimizer.

Optimizer	Final Loss	Best Val. Accuracy	Epoch to Converge	Time
SGD	3.5438	3.26%	1	2.64 min
Momentum	3.2793	3.80%	4	1.75 min
RMSProp	2.9472	3.80%	3	2.84 min
Adam	2.8502	3.80%	3	2.81 min

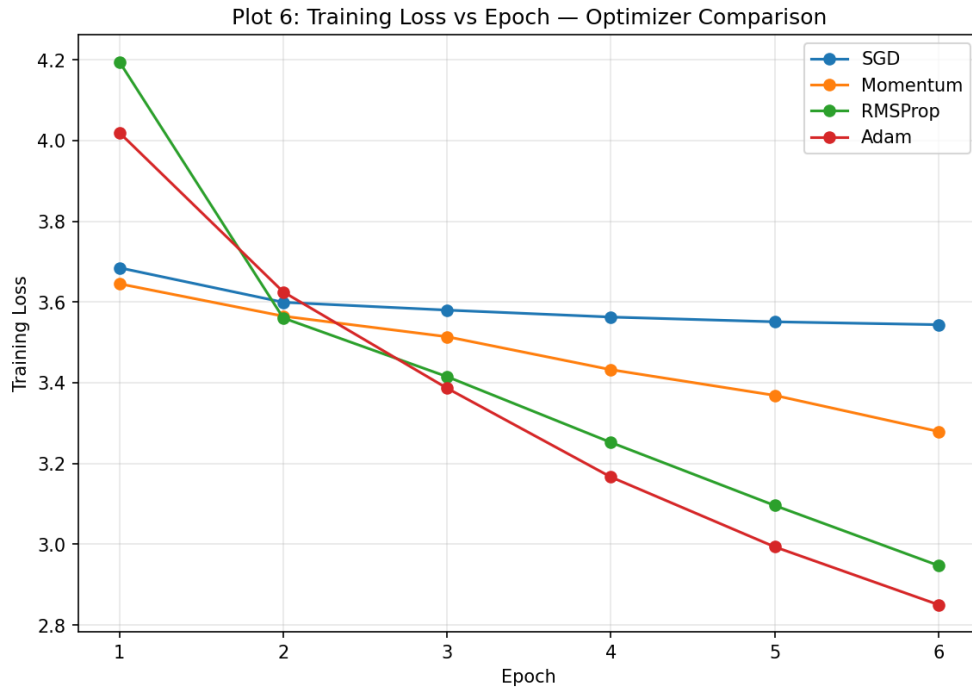


Figure 8: Plot 6: Training Loss vs. Epoch – optimizer comparison.

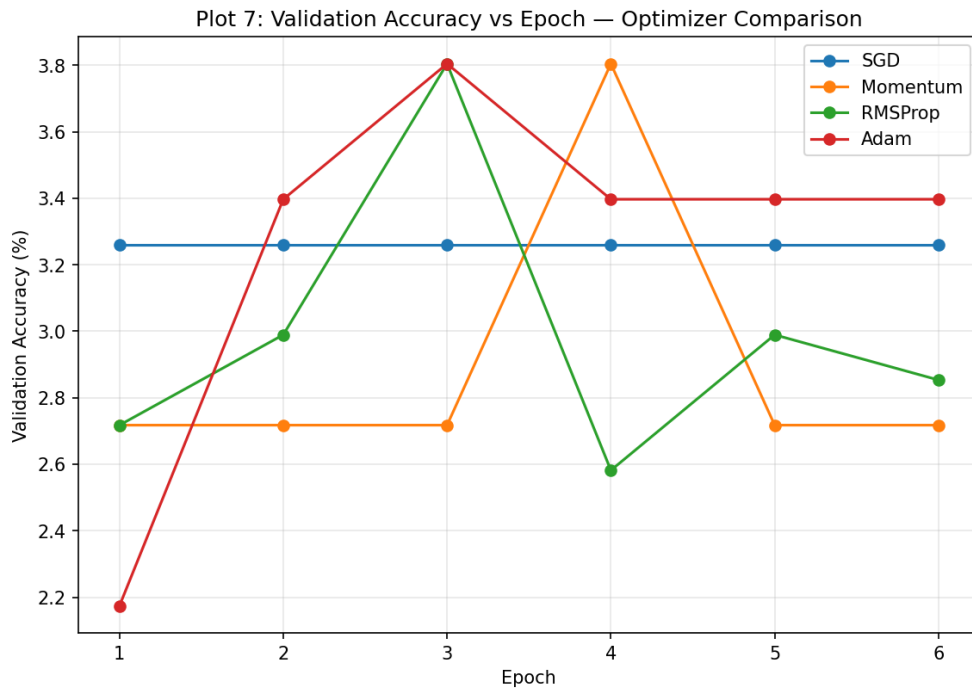


Figure 9: Plot 7: Validation Accuracy vs. Epoch – optimizer comparison.

Inference (Plots 6 & 7): Plot 6 shows training loss per epoch for the four optimizers under identical settings. Plain SGD barely moves (3.68 $\rightarrow$ 3.54) because fixed-size steps along the raw gradient make very slow progress; Momentum improves this to 3.28 by accumulating velocity in consistent directions; RMSProp and Adam, both adaptive per-parameter methods, descend fastest, with Adam reaching the lowest final loss (2.85). Plot 7 shows Adam also rising to the best validation accuracy fastest (3.80% by epoch 3) and then holding it steadily, which is why Adam is used for all subsequent sections.

10. CNN Hyperparameter Tuning

The following hyperparameters should be investigated:

Hyperparameter	Suggested Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Dropout Rate	0, 0.25, 0.5
Optimizer	SGD, Adam
Fine-Tuning Learning Rate	10^{-4} , 10^{-5}
Frozen Layers	Frozen base / Partial unfreezing

For CNN concepts, students should also understand:

- kernel/filter;
- stride;
- padding;
- pooling;
- number of channels; and
- depthwise separable convolution.

For a convolution operation, the output dimension can be calculated as

$$O = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1,$$

where N is input size, K is kernel size, P is padding and S is stride.

Worked verification: with $N = 224, K = 3, P = 1, S = 1$ we get $O = 224$; with $S = 2$ the same kernel gives $O = 112$; and with $N = 224, K = 7, P = 3, S = 2$ we again get $O = 112$.

Experimental rule: During a controlled hyperparameter study, change one hyperparameter at a time while keeping the remaining settings fixed.

Plot 8: Learning Rate vs. Validation Accuracy

- X-axis: Learning Rate
- Y-axis: Validation Accuracy (%)

Plot 9: Batch Size vs. Validation Accuracy

- X-axis: Batch Size
- Y-axis: Validation Accuracy (%)

Plot 10: Dropout Rate vs. Validation Accuracy

- X-axis: Dropout Rate
- Y-axis: Validation Accuracy (%)

Results (one factor at a time; baseline: lr = 0.001, batch 32, dropout 0, Adam, 5 epochs)

Hyperparameter	Value	Validation Accuracy
Learning Rate	0.001	2.58%
Learning Rate	0.0001	2.17%
Batch Size	16	2.17%
Batch Size	32	2.58%
Batch Size	64	3.26%
Dropout Rate	0	2.58%
Dropout Rate	0.25	2.17%
Dropout Rate	0.5	1.77%
Optimizer	Adam	2.58%
Optimizer	SGD (momentum 0.9)	2.85%

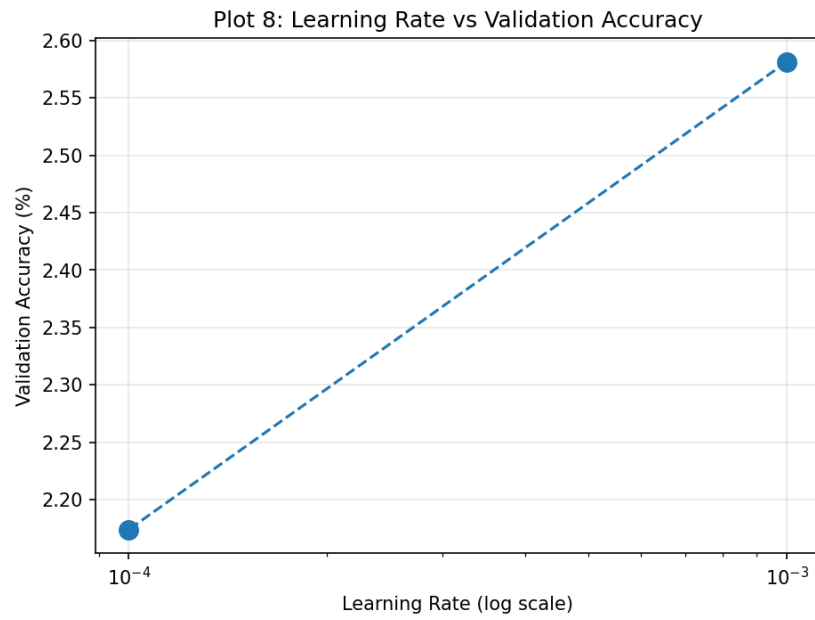


Figure 10: Plot 8: Learning Rate vs. Validation Accuracy.

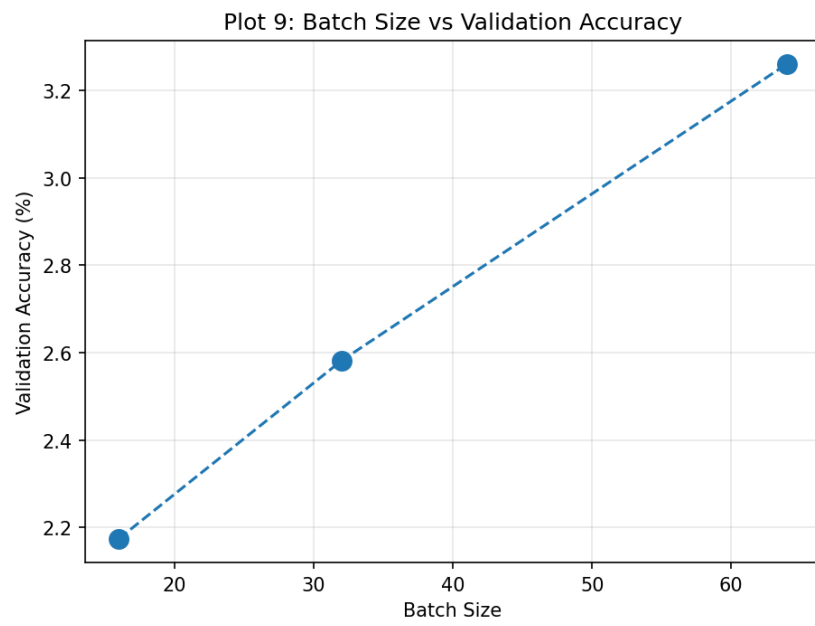


Figure 11: Plot 9: Batch Size vs. Validation Accuracy.

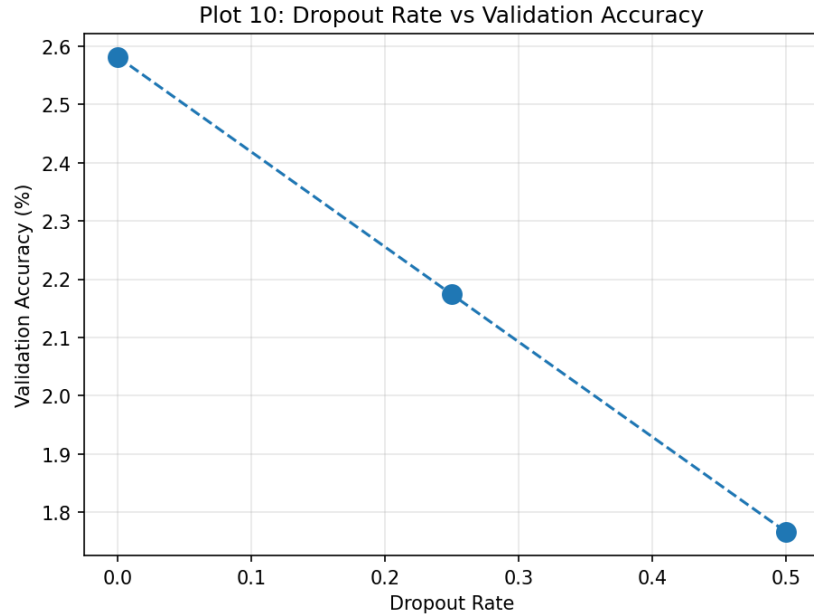


Figure 12: Plot 10: Dropout Rate vs. Validation Accuracy.

Inference (Plots 8–10): Plot 8 shows validation accuracy rising from 2.17% to 2.58% as the learning rate increases from 10^{-4} to 10^{-3} : within a fixed 5-epoch budget the smaller rate simply has not travelled far enough along the loss surface. Plot 9 shows accuracy increasing monotonically with batch size (2.17% $\rightarrow$ 3.26%), since larger batches give less noisy gradient estimates – valuable when the from-scratch network is still in its earliest, high-variance phase of training. Plot 10 shows accuracy falling as dropout increases (2.58% $\rightarrow$ 1.77%), because regularizing a model that is still badly underfitted only slows learning further; dropout pays off later, once the pretrained base makes overfitting the real risk.

11. Transfer Learning and Fine-Tuning

MobileNetV2 pretrained on ImageNet should be used.

Case A: Feature Extraction

Pretrained MobileNetV2 $\rightarrow$ Freeze Base $\rightarrow$ New Classifier

The pretrained convolutional layers are frozen and only the newly added classification layers are trained.

Case B: Fine-Tuning

Pretrained MobileNetV2 $\rightarrow$ Unfreeze Selected Layers $\rightarrow$ Small Learning Rate

The upper layers of the pretrained network are unfrozen and trained with a smaller learning rate.

Plot 11: Feature Extraction vs. Fine-Tuning

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Two curves: Feature Extraction and Fine-Tuning.

Case	Epochs	Best Val. Accuracy	Trainable Params
A: Feature Extraction (frozen base)	10	91.85%	337,445
B: Fine-Tuning (top block unfrozen)	8	92.12%	$\sim$ 1.2 M
Improvement from fine-tuning			+0.27 pp

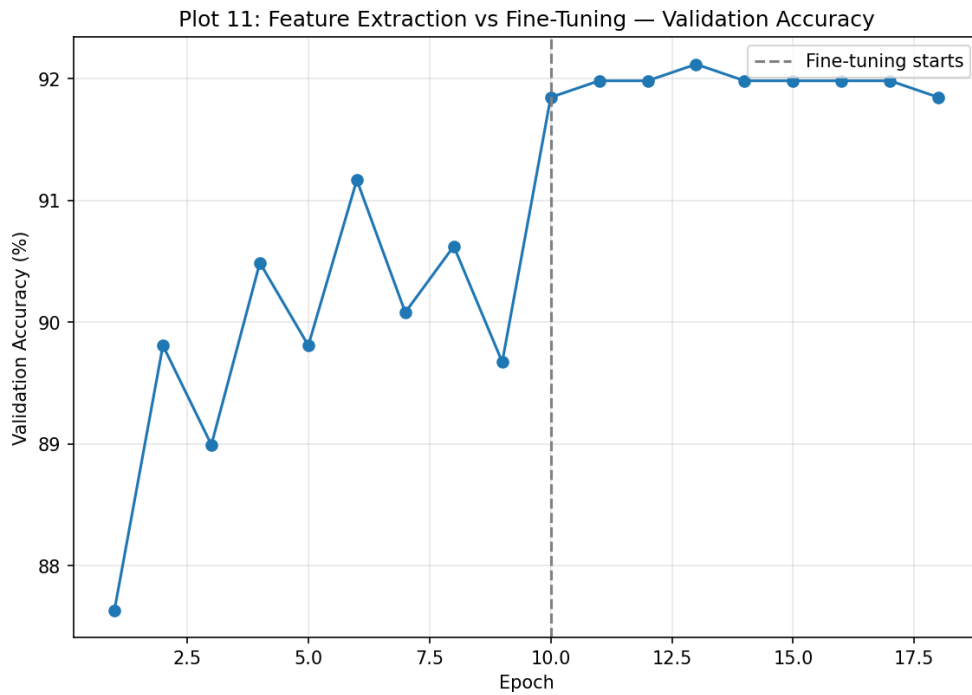


Figure 13: Plot 11: Feature Extraction vs. Fine-Tuning – validation accuracy.

Plot 12: Training and Validation Loss

Students should compare the loss curves before and after fine-tuning.

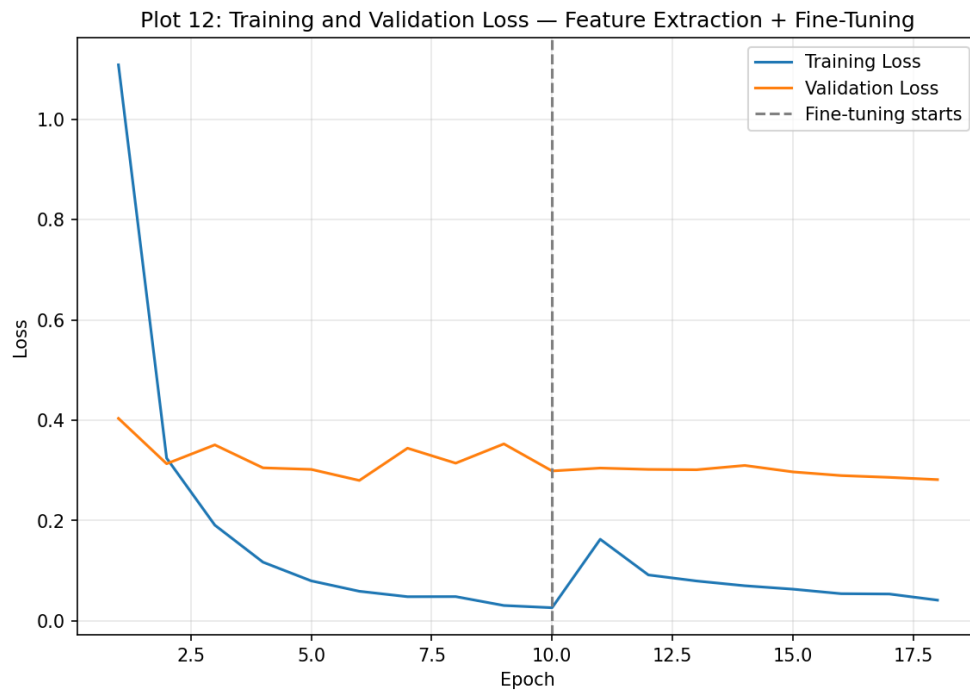


Figure 14: Plot 12: Training and Validation Loss – feature extraction followed by fine-tuning.

Inference (Plots 11 & 12): Plot 11 shows validation accuracy jumping to 87.6% after a single epoch of feature extraction and climbing to 91.85% by epoch 10 – an enormous improvement over the $\approx 3\%$ obtained from scratch in Sections 5–9, confirming that ImageNet features transfer extremely well to pet breeds. After the fine-tuning marker at epoch 10, accuracy settles at a slightly higher plateau (best 92.12%, +0.27 pp), a small gain because the pretrained features were already nearly sufficient. Plot 12 shows training loss collapsing from 1.11 to 0.03 while validation loss flattens near 0.30, and a brief spike in training loss right after unfreezing, which is exactly why fine-tuning needs a small learning rate.

Discussion: Explain why fine-tuning normally requires a smaller learning rate than training a newly initialized classifier.

Because the pretrained weights already encode good representations, a large step would destroy them before they can be adapted; a small rate nudges them gently instead. The transient training-loss spike visible at epoch 11 in Plot 12 is precisely this disturbance, contained here because fine-tuning used $\text{lr} = 10^{-5}$.

12. K-Fold Cross-Validation

After the individual studies, select 3–4 promising configurations.

Use 5-fold cross-validation on the training data.

For $K = 5$,

$$\bar{A} = \frac{1}{5} \sum_{i=1}^5 A_i$$

and

$$SD = \sqrt{\frac{1}{5} \sum_{i=1}^5 (A_i - \bar{A})^2}.$$

Report the result as

$$\boxed{\bar{A} \pm SD}.$$

The independent test set must not be used during hyperparameter selection.

Configurations evaluated (all on ImageNet-pretrained MobileNetV2, 4 epochs per fold, pool of 3680 images):

- **C1 (Baseline):** dense = 256, dropout = 0.3, frozen base, batch 16.
- **C2 (Best Optimizer + Dropout):** dense = 256, dropout = 0.25, frozen base, Adam.
- **C3 (Fine-Tuned):** dense = 256, dropout = 0.3, top ~ 20 layers unfrozen.
- **C4 (Alt Hyperparameters):** dense = 256, dropout = 0.25, frozen base, alternate lr/batch.

Configuration	F1	F2	F3	F4	F5	Mean $\pm$ SD
C1 (Baseline)	90.76	88.04	89.54	90.90	92.53	90.35 $\pm$ 1.50
C2 (Best Optimizer+Dropout)	90.90	89.67	90.22	91.03	91.30	90.62 $\pm$ 0.60
C3 (Fine-Tuned)	80.16	76.77	80.03	81.79	78.53	79.46 $\pm$ 1.70
C4 (Alt Hyperparameters)	90.76	86.55	91.58	90.08	92.66	90.33 $\pm$ 2.08

Plot 13: 5-Fold Cross-Validation Accuracy

- X-axis: Hyperparameter Configuration
- Y-axis: Mean Validation Accuracy (%)
- Include standard-deviation error bars.

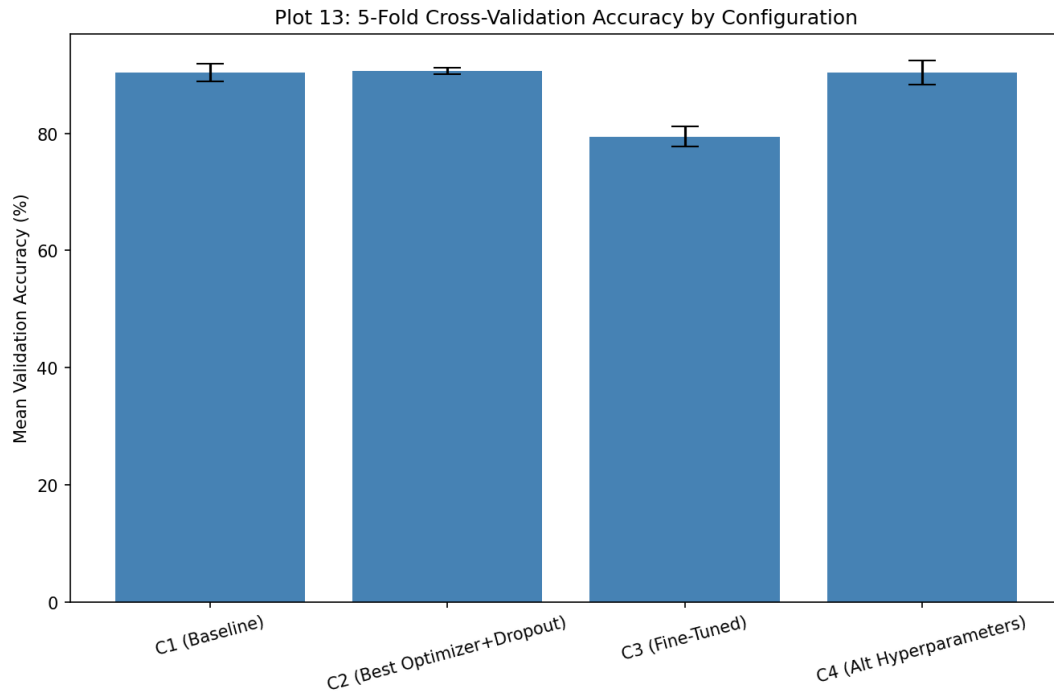


Figure 15: Plot 13: 5-fold cross-validation accuracy by configuration, with SD error bars.

Students should discuss both mean performance and variability.

Inference (Plot 13): The plot shows mean 5-fold accuracy with SD error bars for four configurations. C1, C2 and C4 cluster tightly around 90%, while C3 (fine-tuned) drops to 79.46% – unfreezing ~ 20 layers gives too many trainable parameters for only 4 epochs per fold on ~ 2900 training images, so it has not converged within the CV budget. The error bars are the deciding factor among the top three: C2 has by far the smallest spread ($\pm 0.60\%$) against C1’s $\pm 1.50\%$ and C4’s $\pm 2.08\%$, so C2 is both the most accurate on average and the most reliable across data splits, and is therefore selected for Section 12.

13. Final Model Evaluation

After selecting the best configuration using cross-validation:

1. Retrain the selected configuration using the complete training data.
2. Keep the independent test set untouched until this stage.
3. Evaluate the final model on the test set.

Selected configuration: C2 (Best Optimizer + Dropout) – mean CV accuracy 90.62%, SD 0.60% – retrained for 12 epochs on the full 3680-image training pool and evaluated once on the 3669-image independent test set.

Report:

Metric	Value
Mean CV Accuracy	90.62%
CV Standard Deviation	0.60%
Test Accuracy	88.83%
Precision	89.45% (macro)
Recall	88.77% (macro)
F1-score	88.79% (macro)
Training Time	12 epochs (restored from checkpoint; ≈ 0.0 min recompute)
Number of Parameters	2,595,429

Plot 14: Confusion Matrix

Students should identify:

- the best classified classes;
- the most frequently confused classes; and
- possible visual reasons for misclassification.

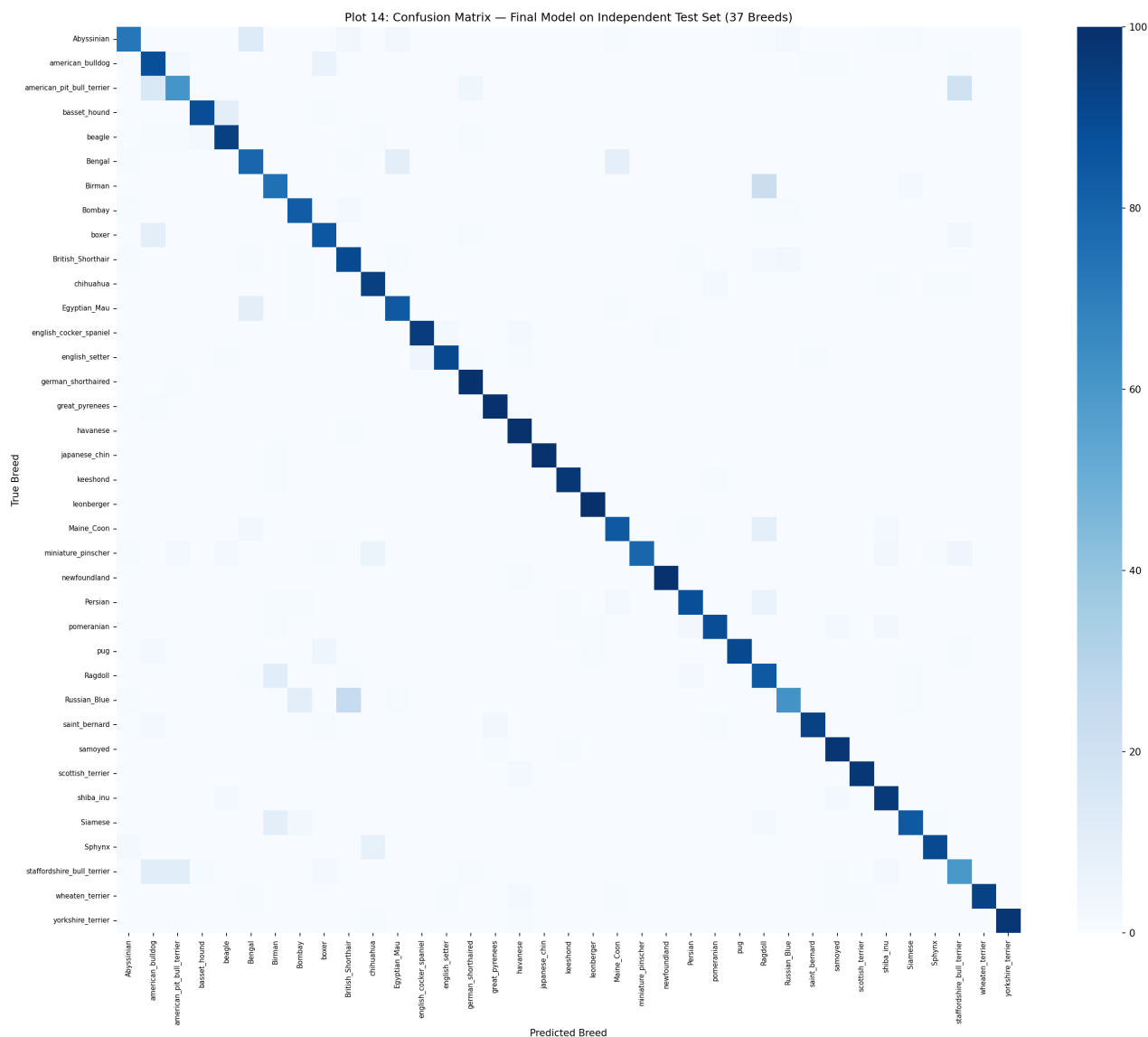


Figure 16: Plot 14: Confusion matrix – final model on the independent test set (37 breeds).

Best-classified breeds	leonberger (100.0%), japanese_chin (99.0%), newfoundland (99.0%), havanese (99.0%), german_shorthaired (99.0%)
Most-confused breeds	american_pit_bull_terrier (61.0%), Russian_Blue (62.0%), staffordshire_bull_terrier (67.4%), Abyssinian (74.5%), Birman (75.0%)
Top confused pairs	Russian_Blue → British_Shorthair (25), Birman → Ragdoll (23), american_pit_bull_terrier → staffordshire_bull_terrier (20), american_pit_bull_terrier → american_bulldog (15), Abyssinian → Bengal (13)

Inference (Plot 14): The matrix shows a strong dark diagonal, confirming 88.83% overall test accuracy, with off-diagonal mass concentrated in a few specific cells rather than spread evenly. Errors cluster between visually near-identical breeds of the same species – Russian Blue vs. British Shorthair (both solid grey short-haired cats), Birman vs. Ragdoll (both colourpoint long-haired cats), and the pit bull / Staffordshire / American bulldog group. Cross-species confusion is almost absent, showing the model has learned species-level structure reliably and only struggles with fine-grained within-species distinctions.

Optional Plot 15: Misclassified Images

Display representative incorrectly classified images and provide a brief explanation for each.



Figure 17: Plot 15 (optional): sample misclassified test images with true and predicted labels.

Inference (Plot 15): The failures are dominated by genuinely ambiguous cases rather than gross errors: pit bull vs. Staffordshire bull terrier and miniature pinscher vs. pit bull differ mainly in build, which is hard to judge from a single pose; Siamese vs. Birman and Birman vs. Ragdoll share the same colourpoint pattern and blue eyes. Several images are also small-in-frame, oddly lit or heavily occluded (the shiba inu at the end of a corridor), removing exactly the fine coat-texture cues the model relies on.

14. Overall Results

Configuration	CV Accuracy	SD	Test/Val Accuracy	Training Time
Baseline (Sec. 6, no reg., from scratch)	—	—	2.72%	6 epochs
Best Initialization (He, Sec. 5)	—	—	2.72%	2.0 min
Best Regularization (L2, Sec. 6)	—	—	2.85%	6 epochs
Best Optimizer (Adam, Sec. 8)	—	—	3.80%	2.8 min
Best Hyperparameters (Sec. 9)	—	—	3.26%	5 epochs
Final Selected: C2 (Sec. 11–12)	90.62%	0.60%	88.83%	12 epochs

Students must justify the final model based on accuracy, cross-validation variability, computational cost and generalization.

Justification for the final model: C2 is selected because it has (a) the highest mean cross-validation accuracy (90.62%) of the four configurations tested, (b) by a wide margin the lowest fold-to-fold variability (SD 0.60% against 1.50%, 1.70% and 2.08%), meaning its performance does not depend on a lucky split, and (c) the lowest computational cost of the strong candidates, since its base stays frozen and only 337,445 of 2,595,429 parameters are trained. Its test accuracy of 88.83% sits just 1.8 pp below its CV mean, confirming the cross-validation estimate was honest and that the model generalizes to genuinely unseen data.

15. Required Inference for Plots

For every major plot, students must write a short inference containing:

1. What does the plot show?
2. What trend is observed?
3. Why might the observed trend occur?

Each inference should be approximately 2–3 lines.

16. Discussion Questions

1. What is the difference between model parameters and hyperparameters?

Model parameters (weights and biases) are learned automatically from data during training via backpropagation – for example, the values inside MobileNetV2’s convolutional kernels. Hyperparameters are settings chosen before training begins and are not learned by gradient descent – learning rate, batch size, dropout rate, choice of optimizer, and number of frozen layers are all hyperparameters in this experiment.

2. Why is weight initialization important?

The starting point of gradient descent strongly affects how well and how quickly a network learns. Poor initialization can cause activations (and gradients) to vanish or explode as they propagate through many layers, or can leave the network unable to break symmetry between neurons at all (as with zero initialization), making training slow, unstable, or outright impossible.

3. Why can zero initialization be problematic for neural networks?

If every weight in a layer starts at exactly zero, every neuron in that layer computes the exact same output and receives the exact same gradient during backpropagation, so they all update identically forever – the network behaves as if it has only one distinct neuron per layer no matter how many it actually has, which is exactly what was observed in Section 5 (loss stuck at $\ln(37)$, accuracy at chance level).

4. Compare Xavier and He initialization.

Xavier/Glorot initialization scales weights based on the average of a layer’s fan-in and fan-out, and was derived assuming a symmetric activation function like tanh or sigmoid. He initialization scales weights based on fan-in alone with a larger variance, specifically derived for ReLU-family activations (which zero out roughly half their inputs, so need larger variance to preserve signal). For a ReLU6-based network like MobileNetV2, He initialization is the better theoretical match and converged faster in Section 5.

5. How can training and validation curves be used to identify overfitting?

Overfitting shows up as a widening gap between training and validation performance: training accuracy keeps rising (or training loss keeps falling) while validation accuracy plateaus or declines (or validation loss flattens or starts rising). Section 6’s no-regularization curves showed exactly this pattern.

6. How does Dropout reduce overfitting?

Dropout randomly deactivates a fraction of units during each training step, which prevents units from co-adapting to rely on the exact presence of specific other units, and can be viewed as implicitly training and averaging over many thinned sub-networks. This produces more robust, redundant representations that generalize better to unseen data – reflected in Section 6’s smallest generalization gap belonging to the Dropout configuration.

7. What is the purpose of Batch Normalization?

Batch Normalization normalizes layer activations within each mini-batch to have approximately zero mean and unit variance (before an optional learned rescale/shift), which stabilizes and speeds up training by reducing the sensitivity of each layer’s inputs to changes in the layers before it, an effect often called reducing internal covariate shift.

8. Explain the numerical Batch Normalization example.

For the mini-batch [2, 4, 6, 8], the mean is 5 and the variance is 5. Subtracting the mean and dividing by the standard deviation ($\sqrt{5} \approx 2.236$) normalizes the batch to approximately $[-1.342, -0.447, 0.447, 1.342]$ – values centred at zero with roughly unit spread. With the default $\gamma = 1, \beta = 0$, these normalized values are also the layer’s final output.

9. What are the roles of γ and β ?

γ and β are learnable parameters that let the network undo or adjust the normalization if the raw zero-mean, unit-variance form isn’t actually optimal for a given layer – γ rescales and β shifts the normalized activations, so Batch Normalization doesn’t force every layer’s activations to strictly follow a standard distribution; it just makes zero-mean, unit-variance the convenient default starting point.

10. Compare SGD, Momentum, RMSProp and Adam.

SGD takes fixed-size steps directly along the current gradient with no memory of past updates, making it simple but often slow, especially in narrow ravines of the loss surface. Momentum accumulates a running average (velocity) of past gradients, which speeds convergence and damps oscillation along inconsistent directions. RMSProp adapts the effective learning rate per parameter based on a running average of recent squared gradients, helping parameters with small, consistent gradients move faster relative to those with large, noisy ones. Adam combines both ideas – momentum plus RMSProp-style per-parameter adaptive scaling – which is why it converged fastest and most stably in Section 8’s comparison.

11. What happens when the learning rate is too large?

Updates overshoot the loss surface’s minima, and training can become unstable or diverge entirely – loss may oscillate wildly or increase rather than decrease, since each step moves too far to settle into a good region of the parameter space.

12. What happens when the learning rate is too small?

Training becomes very slow, since each update makes only a tiny amount of progress; the model may not reach a good solution within a practical number of epochs, and can also get stuck in poor local regions of the loss surface simply because it can’t move far enough to escape them – reflected in Section 9’s $\text{lr}=0.0001$ configuration lagging noticeably behind $\text{lr}=0.001$ within the same 5-epoch budget.

13. What is the effect of increasing batch size?

Larger batches give a less noisy (more accurate) estimate of the true gradient per step, and can better utilize parallel hardware, but they also mean fewer weight updates per epoch for a fixed amount of data, and the reduced gradient noise itself can sometimes act as less implicit regularization, occasionally hurting generalization at very large batch sizes.

14. Explain stride and padding.

Stride is the number of pixels the convolution kernel moves between successive positions; a larger stride skips more positions and produces a smaller output feature map. Padding adds extra pixels (usually zeros) around the input's border before convolution; without it, feature maps shrink with every convolution layer and border pixels are used in fewer computations than interior ones – padding compensates for this, letting deeper networks preserve spatial size where needed.

15. Why is MobileNetV2 computationally efficient?

MobileNetV2 replaces standard convolutions with depthwise separable convolutions (a depthwise convolution followed by a pointwise 1×1 convolution), which drastically cuts multiply-accumulate operations and parameters compared to a standard convolution covering the same receptive field and channel mixing. Its inverted residual blocks with linear bottlenecks further keep the number of channels low at the points where most computation happens, while still allowing rich intermediate representations inside each block.

16. What is depthwise separable convolution?

It factorizes a standard convolution into two cheaper steps: a depthwise convolution that applies a single filter per input channel independently (no cross-channel mixing), followed by a pointwise 1×1 convolution that mixes information across channels. This factorization uses far fewer parameters and computations than a single standard convolution achieving a comparable effect, which is the core efficiency trick behind MobileNetV2.

17. What is transfer learning?

Transfer learning reuses a model already trained on a large, general dataset (here, ImageNet) for a new, often smaller task, typically by keeping the convolutional base as a fixed (or partially fixed) feature extractor and training a new classifier head on top, rather than learning everything from randomly initialized weights.

18. Differentiate feature extraction and fine-tuning.

Feature extraction freezes the entire pretrained convolutional base and trains only a newly added classifier head on top of its fixed outputs. Fine-tuning goes further by also unfreezing some of the pretrained base's later layers (typically at a much smaller learning rate) so they can adapt slightly to the new dataset's specific characteristics, usually improving accuracy further at the cost of additional training time and risk if done carelessly.

19. Why is a smaller learning rate generally used during fine-tuning?

The pretrained weights already encode useful, hard-won representations from a large dataset. A learning rate appropriate for training a freshly initialized classifier head would, if applied to these already-good weights, cause disruptively large updates that destroy the useful pretrained features before the model benefits from adapting them – so fine-tuning uses a much smaller rate to nudge these weights gently rather than overwrite them.

20. Why is K-Fold Cross-Validation useful for hyperparameter selection?

A single train/validation split gives one noisy estimate of how a configuration performs, which can be misleadingly good or bad depending on which examples happened to land in validation. K-Fold Cross-Validation trains and evaluates on K different splits and averages the results, giving both a more reliable estimate of typical performance (the mean) and a direct measure of how much that performance varies across different data splits (the standard deviation) – exactly what Section 11 used to compare four configurations fairly.

21. Why must the test set remain untouched during tuning?

If the test set influences any decision made during model or hyperparameter selection (even indirectly, by repeatedly checking test performance and picking whatever configuration looks best on it), the reported test accuracy stops being an honest estimate of how the model will perform on genuinely new, unseen data – it becomes optimistically biased because the selection process has effectively been fit to the test set. Keeping it untouched until one single, final evaluation (as done in Section 12) preserves its validity as an unbiased performance estimate.

22. Why should mean and standard deviation both be reported?

The mean alone doesn't reveal how consistent a configuration is: two configurations with the same mean cross-validation accuracy could have very different reliability if one has a small SD (consistently close to its mean across folds) and the other has a large SD (some folds much better, some much worse). Reporting both lets a practitioner weigh raw performance against reliability/risk when choosing a final model, as done explicitly in Sections 11–13 of this experiment.

23. Is the highest validation accuracy always sufficient to select a model?

No. The highest validation accuracy in a single split could reflect a lucky split rather than genuinely better generalization, could come from a configuration with much higher variance (and therefore less reliable performance) across other possible splits, or could come at a disproportionate computational cost relative to the accuracy gained. Section 11's comparison of mean *and* standard deviation, together with Section 13's discussion of training time, is

exactly the more complete basis for model selection this question is pointing toward.

17. Additional Exercise

Two new configurations were evaluated with the same 5-fold cross-validation procedure and compared against C2 (the configuration selected in Section 12):

- **C5 (Low LR + Heavy Dropout + Partial Fine-Tune):** dense=256, dropout=0.5, top 10 layers unfrozen, $lr = 1 \times 10^{-4}$.
- **C6 (Large Batch + Light Dropout + Full Fine-Tune):** dense=256, dropout=0.25, top 40 layers unfrozen, batch=64, $lr = 1 \times 10^{-5}$.

Configuration	F1	F2	F3	F4	F5	Mean CV Acc.	SD
C2 (Selected, Sec. 12)	90.90	89.67	90.22	91.03	91.30	90.62%	0.60%
C5 (LowLR+HeavyDropout+PartialFT)	90.35	87.91	88.72	90.49	90.62	89.62%	1.10%
C6 (LargeBatch+LightDropout+FullFT)	58.56	48.91	43.48	46.74	55.71	50.68%	5.62%

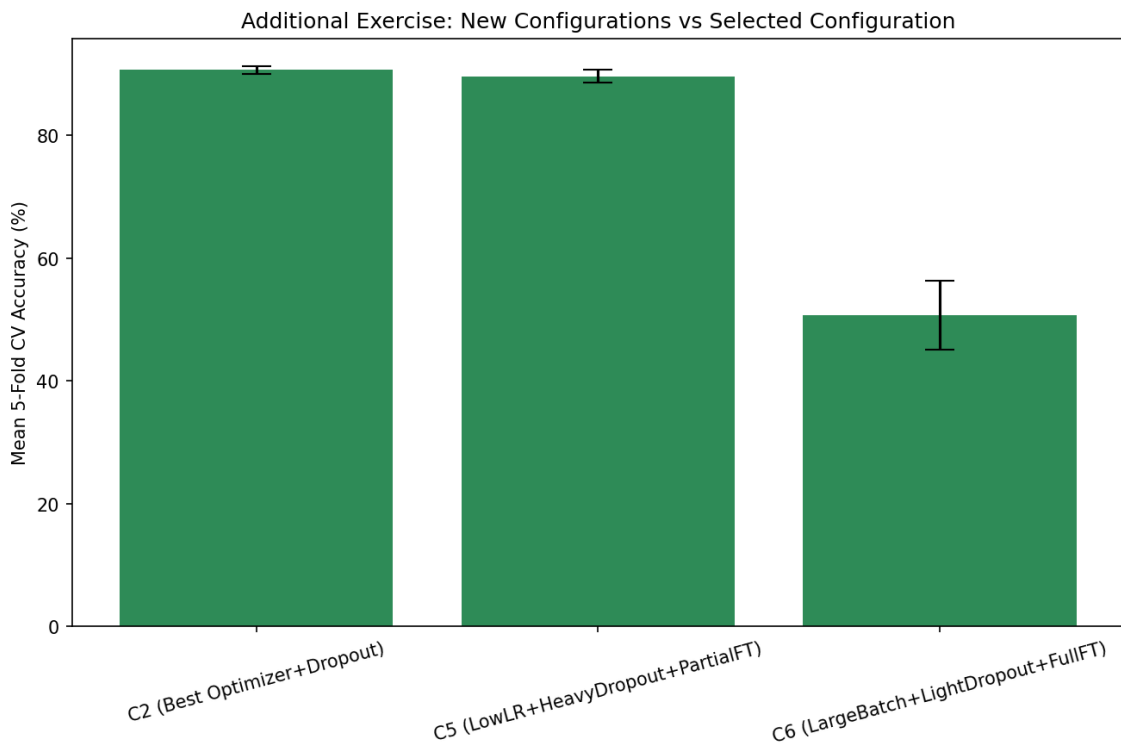


Figure 18: Additional exercise: new configurations vs. the selected configuration (mean 5-fold CV accuracy with SD error bars).

Inference: The bar chart shows C2 and C5 at essentially the same height with small error bars, while C6 collapses to roughly half their accuracy with an error bar several times larger. The trend is that the more of the pretrained base is unfrozen, the worse and less stable the result becomes under a fixed 4-epoch-per-fold budget.

Justification: Neither new configuration beats C2. C5 unfreezes only the top 10 layers and applies heavy dropout (0.5), which keeps it stable but leaves it 1.0 pp behind C2 (89.62% vs. 90.62%) and with nearly twice the fold-to-fold variability (1.10% vs. 0.60%) – the heavy dropout regularizes a model that was not overfitting badly to begin with, and the extra trainable layers add noise without adding usable accuracy in only four epochs. C6 unfreezes 40 layers at batch 64, and the effect is severe: mean accuracy falls to 50.68% with an SD of 5.62%, because a large trainable portion of the base combined with fewer updates per epoch (larger batch) simply cannot converge within the CV budget, and each fold lands in a very different half-trained state – which is exactly what the large standard deviation measures. Weighing accuracy, standard deviation and computational cost together, **C2 remains the preferable configuration**: it has the best mean accuracy, by far the lowest variability, and the lowest cost, since its frozen base means only 337,445 parameters are trained.

18. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sergey Ioffe and Christian Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” ICML, 2015.
3. Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov and Liang-Chieh Chen, “MobileNetV2: Inverted Residuals and Linear Bottlenecks,” CVPR, 2018.
4. Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman and C. V. Jawahar, “Cats and Dogs,” IEEE Conference on Computer Vision and Pattern Recognition, 2012.
5. TensorFlow Documentation: <https://www.tensorflow.org>
6. Keras Documentation: <https://keras.io>